

BigQuery Activity & Cost Report

Project **project-dw** · region **region-eu** · last 30 days
Generated 2026-09-03 15:17:42 by Cloud Crucix as healthcheck@project-dw.iam.gserviceaccount.com

Summary	Value
Jobs analysed	27,076
Bytes billed	45.1 TB
Estimated query cost	\$281.57
Slot hours consumed	774.2
Served from cache (billed nothing)	405 jobs, 0.0 B spared
Spent on jobs that failed	\$0.0115 across 188 jobs
Storage, per month	\$7.85 over 574 tables

Query cost uses \$6.25 per TiB (built-in default (billing catalog unavailable: Cloud Billing API has not been used in project 000000000000 before or it is disabled. Enable it by visiting the API console for project 000000000000, then retry. If you enabled this API recently, wait a few minutes for the action to propagate to our systems and retry.)). Costs are estimates from bytes billed before the 1 TiB/month free tier; the invoice is authoritative.

Surprise-bill findings

[HIGH] R01 · SELECT * over a large scan

633 occurrence(s) · 4.0 TB scanned · \$24.72
633 queries selected every column over 1 GB or more. BigQuery bills for the columns a query reads, not the rows it returns, so SELECT * pays for the whole width of the table even when the result is one row.
What to do: Name the columns you need. On a wide table this is often a 10x reduction on its own.
user1@example.com · 31.60 GB
with ddd as (select LegalUnitNo, IdBusinessKey, Source, count(*) from
project-legacy.legacy_dw_prod.fact_RevenueAmounts group by 1,2,3 having count(*)>1) select * from
project-legacy.legacy_dw
user3@example.com · 25.25 GB
select * from `region-eu`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
dbt-runner@project-dw.iam.gserviceaccount.com · 17.40 GB
create or replace table `project-dw`.`dw\_prod\_stg`.`stg\_FinancialAmounts` OPTIONS(description=""Contains the
accounts from all sources"") as (with dim_LegalUnit as (select * from `project-dw`.

[MEDIUM] R06 · The same query billed again and again

42 occurrence(s) · 40.9 TB scanned · \$255.33
42 queries ran at least 10 times and billed 41832.9 GB in total. BigQuery's result cache only helps while the underlying data is unchanged, so these re-runs paid full price.
What to do: Materialise the result on a schedule - a table or a materialised view - and point the readers at that instead of re-scanning the sources every time.
dbt-runner@project-dw.iam.gserviceaccount.com · 2,638 runs · 31496.43 GB
with table_shards_stage as (select info_tables.table_catalog as table_catalog, info_tables.table_schema as
table_schema, coalesce(REGEXP_EXTRACT(info_tables.table_name, ?), info_tables.table_name) as
dbt-runner@project-dw.iam.gserviceaccount.com · 56 runs · 3064.63 GB
BEGIN TRANSACTION;
dbt-runner@project-dw.iam.gserviceaccount.com · 60 runs · 2608.18 GB
MERGE `project-dw`.`dw\_prod`.`fact\_FinancialAmounts` AS T USING (select LegalUnitNo, JNo, EntNo, DateKey,
CurrencyISO, AccountsSID, Gr1, Gr2, Gr7, CustSID, SuppSID, DepartmentSID, Pro

[MEDIUM] R02 · Heavy query with no LIMIT

2,214 occurrence(s) · 34.8 TB scanned · \$217.28
2214 queries scanned 10 GB or more with nothing capping the result. Exploratory queries without a LIMIT are the usual way a quick look becomes an expensive one.
What to do: Add LIMIT while exploring. It caps the result, and with a partition filter it caps the scan too.
dbt-runner@project-dw.iam.gserviceaccount.com · 105.23 GB
BEGIN TRANSACTION;
dbt-runner@project-dw.iam.gserviceaccount.com · 96.18 GB
BEGIN TRANSACTION;
dbt-runner@project-dw.iam.gserviceaccount.com · 94.86 GB
BEGIN TRANSACTION;

[MEDIUM] R05 · Query mixes tables from inside and outside region-eu

2,796 occurrence(s) · 4.6 TB scanned · \$28.61

2796 queries referenced tables that are not in region-eu alongside tables that are. That means either a cross-region read - which moves data between locations and bills at the pricier side - or another project, which makes the spend hard to attribute.

What to do: Co-locate the tables, or materialise a copy of the smaller one next to the larger. Confirm the location first: a table in another project of the same region also lands here.

```
dbt-runner@project-dw.iam.gserviceaccount.com · 17.40 GB
create or replace table `project-dw`.`dw_prod_stg`.`stg_FinancialAmounts` OPTIONS( description=""Contains the
accounts from all sources"" ) as ( with dim_LegalUnit as ( select * from `project-dw`.
dbt-runner@project-dw.iam.gserviceaccount.com · 17.40 GB
create or replace table `project-dw`.`dw_prod_stg`.`stg_FinancialAmounts` OPTIONS( description=""Contains the
accounts from all sources"" ) as ( with dim_LegalUnit as ( select * from `project-dw`.
dbt-runner@project-dw.iam.gserviceaccount.com · 17.38 GB
create or replace table `project-dw`.`dw_prod_stg`.`stg_FinancialAmounts` OPTIONS( description=""Contains the
accounts from all sources"" ) as ( with dim_LegalUnit as ( select * from `project-dw`.
```

[MEDIUM] R04 · Partitioned table read without its partition filter

126 occurrence(s) · 612.1 GB scanned · \$3.74

126 queries read a partitioned table without filtering on its partition column, so BigQuery scanned - and billed for - every partition instead of the few that were needed. This is the most common cause of a surprise bill.

What to do: Filter on the partition column named in each example. For tables where this keeps happening, set `require_partition_filter` so BigQuery refuses the query instead of billing for it.

```
table dw_prod.fact_FinancialAmounts · partition column DateKey · dbt-runner@project-dw.iam.gserviceaccount.com · 9.36 GB
select count(*) as failures, count(*) != 0 as should_warn, count(*) != 0 as should_error from ( with
validation_errors as ( select LegalUnitNo, JNo, EntNo from `project-dw`.`dw_prod`.`fact_FinancialAmount
table dw_prod.fact_FinancialAmounts · partition column DateKey · dbt-runner@project-dw.iam.gserviceaccount.com · 9.36 GB
select count(*) as failures, count(*) != 0 as should_warn, count(*) != 0 as should_error from ( with
validation_errors as ( select LegalUnitNo, JNo, EntNo from `project-dw`.`dw_prod`.`fact_FinancialAmount
table dw_prod.fact_FinancialAmounts · partition column DateKey · dbt-runner@project-dw.iam.gserviceaccount.com · 9.35 GB
select count(*) as failures, count(*) != 0 as should_warn, count(*) != 0 as should_error from ( with
validation_errors as ( select LegalUnitNo, JNo, EntNo from `project-dw`.`dw_prod`.`fact_FinancialAmount
```

[LOW] R03 · Tables costing storage that nobody reads

6 occurrence(s) · 2.9 GB scanned · \$0.0586 per month

6 tables hold 2.9 GB, were last modified over 90 days ago, and were not read once in this window. Storage bills every month whether or not anyone queries it.

What to do: Confirm they are still needed. If they are history, export to Cloud Storage and drop them; if they are cold but wanted, long-term storage already applies at half price after 90 days of no modification.

```
table dbt_dev2.fact_RevenueAmounts · 0.83 GB · last modified 2026-03-05
table dbt_dev2_stg.stg_FinancialAmounts · 0.63 GB · last modified 2026-02-03
table dbt_dev2.fact_FinancialAmounts · 0.63 GB · last modified 2026-02-03
```

[LOW] R07 · Spend on jobs that failed anyway

3 occurrence(s) · 1.9 GB scanned · \$0.0115

3 jobs failed after BigQuery had already scanned 1.9 GB. A job that errors part-way still bills for what it read. Top reasons: stopped (2), invalidQuery (1).

What to do: Fix the recurring failures first - they are pure waste. A `maximum_bytes_billed` ceiling on the offending workloads caps the damage while you do.

```
dbt-runner@project-dw.iam.gserviceaccount.com · 1.41 GB
BEGIN TRANSACTION;
dbt-runner@project-dw.iam.gserviceaccount.com · 0.24 GB
create or replace table `project-dw`.`dbt_cloud_pr_XXXXXX_225_stg`.`stg_Customer` OPTIONS( description="" "" ) as
( with Restricted_LegalUnit as ( select *, ErpCode as ErpCompanyId from `project-dw`.
dbt-runner@project-dw.iam.gserviceaccount.com · 0.23 GB
CALL `project-dw`.dbt_dev1.spLoad_dimDefaults();
```

Where the spend went

People

Person	Jobs	Billed	Cost
user1@example.com	1,564	580.1 GB	\$3.54
user2@example.com	123	79.3 GB	\$0.4840

Person	Jobs	Billed	Cost
user3@example.com	13	35.8 GB	\$0.2186

Service accounts

Service account	Jobs	Billed	Cost
dbt-runner@project-dw.iam.gserviceaccount.com	25,084	44.2 TB	\$276.22
healthcheck@project-dw.iam.gserviceaccount.com	292	182.5 GB	\$1.11

dbt models

Model	Jobs	Billed	Cost
model.dbt_dw.fact_FinancialAmounts	699	4.2 TB	\$26.51
model.dbt_dw.stg_RevenueAmounts	94	1.4 TB	\$8.47
model.dbt_dw.stg_FinancialAmounts	92	1.3 TB	\$7.85
model.dbt_dw.fact_Account_Balance	465	641.2 GB	\$3.91
test.dbt_dw.dbt_utils_unique_combination_of_columns_fact_FinancialAmounts_LegalUnitNo__JNo__EntNo	73	506.9 GB	\$3.09
model.dbt_dw.fact_RevenueAmounts	659	400.0 GB	\$2.44
model.dbt_dw.dim_Customer	450	188.3 GB	\$1.15
model.dbt_dw.stg_AccountBalance	93	141.5 GB	\$0.8638
model.dbt_dw.stg_FactICCust	51	114.8 GB	\$0.7008
model.dbt_dw.stg_CustomerOpenBalance	69	96.3 GB	\$0.5879
test.dbt_dw.dbt_utils_unique_combination_of_columns_stg_FinancialAmounts_LegalUnitNo__JNo__EntNo	90	74.6 GB	\$0.4554
test.dbt_dw.dbt_utils_unique_combination_of_columns_stg_erp1_fact_financial_amounts_LegalUnitNo__JNo__EntNo	85	73.1 GB	\$0.4461
test.dbt_dw.dbt_utils_unique_combination_of_columns_stg_AccountBalance_LegalUnitNo__AccountsSID__DateKey__SubaccountID	83	61.5 GB	\$0.3752
model.dbt_dw.stg_FactICSupp	48	55.2 GB	\$0.3370
model.dbt_dw.stg_Customer	118	40.3 GB	\$0.2460

Tables

Table	Jobs	Billed	Cost	Biggest consumer
dw_prod.fact_FinancialAmounts	356	4.4 TB	\$27.50	dbt-runner@project-dw (99%)
dw_prod.stg.stg_FinancialAmounts	485	3.4 TB	\$20.99	dbt-runner@project-dw (100%)
dw_prod.Restricted_LegalUnit	1,522	3.2 TB	\$20.15	dbt-runner@project-dw (100%)
dw_prod.dim_LegalUnit	2,178	2.4 TB	\$14.98	dbt-runner@project-dw (100%)
dw_prod.dim_Gr1	316	2.3 TB	\$14.42	dbt-runner@project-dw (100%)
dw_prod.dim_Customer	625	2.2 TB	\$13.52	dbt-runner@project-dw (100%)
dw_prod.dim_IntercompanyCode	582	2.1 TB	\$13.15	dbt-runner@project-dw (100%)
dw_prod.dim_Accounts	576	2.1 TB	\$12.86	dbt-runner@project-dw (100%)
dw_prod.dim_ProductLine	253	2.0 TB	\$12.68	dbt-runner@project-dw (100%)
dw_prod.dim_Departments	466	1.8 TB	\$11.55	dbt-runner@project-dw (100%)
dw_prod.dim_Partner	456	1.8 TB	\$11.54	dbt-runner@project-dw (100%)
dw_prod.dim_CompanyProduct	287	1.8 TB	\$11.53	dbt-runner@project-dw (100%)
erp_source_a.account	800	1.5 TB	\$9.58	dbt-runner@project-dw (99%)
erp_source_b.TransactionDetails	303	1.4 TB	\$9.00	dbt-runner@project-dw (100%)
legacy_dw_prod.dim_LegalUnit	579	1.4 TB	\$8.88	dbt-runner@project-dw (95%)

A job's whole scan is counted against every table it referenced, because BigQuery does not report bytes per table. Use this to rank, not to sum.

Query patterns

Statement (comments and literals stripped)	Runs	Billed	Cost
with table_shards_stage as (select info_tables.table_catalog as table_catalog, info_tables.table_schema as table_schema	2,638	30.8 TB	\$192.24

Statement (comments and literals stripped)	Runs	Billed	Cost
dbt model model.dbt_dw.fact_FinancialAmounts	699	4.2 TB	\$26.51
MERGE `project-dw`.`dw_prod`.`fact_FinancialAmounts` AS T USING (select LegalUnitNo, JNo, EntNo, DateKe	60	2.5 TB	\$15.92
dbt model model.dbt_dw.stg_RevenueAmounts	94	1.4 TB	\$8.47
dbt model model.dbt_dw.stg_FinancialAmounts	92	1.3 TB	\$7.85
dbt model model.dbt_dw.fact_Account_Balance	465	641.2 GB	\$3.91
dbt model test.dbt_dw.dbt_utils_unique_combination_of_columns_fact_FinancialAmounts_LegalUnitNo_JNo_EntNo	73	506.9 GB	\$3.09
DELETE FROM `project-dw`.`dw_prod`.`fact_FinancialAmounts` ffa WHERE ffa.LegalUnitNo IN (SELECT DISTINCT	60	492.8 GB	\$3.01
dbt model model.dbt_dw.fact_RevenueAmounts	659	400.0 GB	\$2.44
MERGE `project-dw`.`dbt_dev1`.`fact_FinancialAmounts` AS T USING (select LegalUnitNo, JNo, EntNo, DateKey, Curr	15	253.9 GB	\$1.55
MERGE `project-dw`.`dw_prod`.`fact_FinancialAmounts` AS T USING (SELECT stg.LegalUnitNo, stg.JNo, FROM	60	242.9 GB	\$1.48
dbt model model.dbt_dw.dim_Customer	450	188.3 GB	\$1.15
with ddd as (select LegalUnitNo, IdBusinessKey, Source, count(*) from project-legacy.legacy_dw_prod.fac	8	179.2 GB	\$1.09
dbt model model.dbt_dw.stg_AccountBalance	93	141.5 GB	\$0.8638
dbt model model.dbt_dw.stg_FactICCust	51	114.8 GB	\$0.7008

Day by day

Date	Jobs	Failed	Billed	Cost	
2026-08-04	1	0	68.0 MB	\$0.0004	
2026-08-05	1,001	12	1.2 TB	\$7.65	SPIKE
2026-08-06	1,505	42	893.7 GB	\$5.45	
2026-08-07	949	25	669.6 GB	\$4.09	
2026-08-08	513	0	380.2 GB	\$2.32	
2026-08-09	498	0	366.6 GB	\$2.24	
2026-08-10	592	14	389.6 GB	\$2.38	
2026-08-11	702	30	387.6 GB	\$2.37	
2026-08-12	669	10	366.4 GB	\$2.24	
2026-08-13	722	12	382.5 GB	\$2.33	
2026-08-14	1,644	19	555.9 GB	\$3.39	
2026-08-15	513	0	367.6 GB	\$2.24	
2026-08-16	497	0	364.5 GB	\$2.22	
2026-08-17	1,303	22	469.6 GB	\$2.87	
2026-08-18	693	18	417.7 GB	\$2.55	
2026-08-19	820	44	426.9 GB	\$2.61	
2026-08-20	848	21	506.7 GB	\$3.09	
2026-08-21	1,116	15	527.5 GB	\$3.22	
2026-08-22	579	0	380.7 GB	\$2.32	
2026-08-23	560	0	369.9 GB	\$2.26	
2026-08-24	645	3	376.5 GB	\$2.30	
2026-08-25	1,226	8	2.2 TB	\$13.84	SPIKE
2026-08-26	948	15	3.5 TB	\$22.02	SPIKE
2026-08-27	1,312	13	4.2 TB	\$26.23	SPIKE
2026-08-28	1,058	10	3.5 TB	\$21.74	SPIKE
2026-08-29	789	0	3.2 TB	\$19.95	SPIKE
2026-08-30	787	0	3.3 TB	\$20.69	SPIKE
2026-08-31	1,363	17	4.4 TB	\$27.57	SPIKE
2026-09-01	1,356	5	4.1 TB	\$25.45	SPIKE
2026-09-02	1,985	19	2.9 TB	\$18.29	SPIKE

Date	Jobs	Failed	Billed	Cost	
2026-09-03	2,099	18	4.1 TB	\$25.71	SPIKE

A day is marked SPIKE when it cost more than twice the median day. The median, not the mean, is the baseline, so one very expensive day cannot raise the bar enough to hide itself.

Storage

Storage	Value
Tables	574
Rows	1,113,467,341
Active (logical)	390.4 GB
Long-term (logical)	4.1 GB
Physical, incl. time travel	80.8 GB
Cost per month, logical billing	\$7.85
Cost per month, physical billing	\$3.22

Worth a look: physical (compressed) billing looks about \$4.63/month cheaper than logical for this data. The switch is per dataset and physical billing also charges for time-travel bytes, which are included above - so treat this as a signal to check the big datasets, not a guaranteed saving.

Rates used: \$0.0200/GiB/month active, \$0.0100/GiB/month long-term.

Largest tables by monthly storage cost

Table	Rows	Size	Per month	Last modified
dw_prod.fact_FinancialAmounts	443,783,397	217.4 GB	\$4.35	2026-09-03
dbt_dev1.fact_FinancialAmounts	98,869,565	48.4 GB	\$0.9685	2026-08-13
dw_prod.fact_RevenueAmounts	98,550,596	28.8 GB	\$0.5769	2026-09-03
dbt_dev1.fact_RevenueAmounts	98,489,619	28.8 GB	\$0.5765	2026-08-07
dw_prod.fact_Account_Balance	24,670,984	4.3 GB	\$0.0861	2026-09-03
dw_prod.fact_Account_Balance_bkp_20260903	24,620,292	4.3 GB	\$0.0859	2026-09-03
dbt_dev1.fact_Account_Balance	24,103,642	4.2 GB	\$0.0841	2026-08-18
dw_test.fact_Account_Balance	19,941,050	3.5 GB	\$0.0702	2026-08-18
dw_prod_stg.stg_AccountBalance	20,460,291	3.4 GB	\$0.0690	2026-09-03
dw_test_stg.stg_AccountBalance	19,941,176	3.4 GB	\$0.0672	2026-08-18
dbt_dev1_stg.stg_AccountBalance	19,941,045	3.4 GB	\$0.0672	2026-08-18
dw_prod_stg.stg_AccountBalance_8	19,893,981	3.3 GB	\$0.0670	2026-09-03
dbt_cloud_pr_XXXXXX_189.fact_Account_Balance	19,416,836	3.3 GB	\$0.0654	2026-07-09
dbt_dev3.fact_Account_Balance	18,898,880	3.2 GB	\$0.0637	2026-06-16
dbt_cloud_pr_XXXXXX_189_stg.stg_AccountBalance	19,416,836	3.1 GB	\$0.0626	2026-07-09

By dataset

Dataset	Tables	Size	Per month
dw_prod	52	259.3 GB	\$5.19
dbt_dev1	52	83.2 GB	\$1.66
dw_prod_stg	45	11.1 GB	\$0.2147
dbt_dev1_stg	38	6.8 GB	\$0.1354
dw_test	48	6.2 GB	\$0.1239
dbt_dev3	44	5.7 GB	\$0.1130
dbt_dev3_stg	47	5.0 GB	\$0.0997
dw_test_stg	42	4.9 GB	\$0.0975
dbt_cloud_pr_XXXXXX_189	10	4.1 GB	\$0.0817
dbt_cloud_pr_XXXXXX_189_stg	3	3.9 GB	\$0.0785
dbt_dev2_stg	21	1.9 GB	\$0.0306
dbt_dev2	38	1.9 GB	\$0.0235
dbt_dev4_stg	15	325.0 MB	\$0.0032

Dataset	Tables	Size	Per month
dbt_dev5_stg	18	179.2 MB	\$0.0017
dbt_dev5	33	93.7 MB	\$0.0009
dw_manual_data	24	12.9 MB	\$0.0003
dbt_dev4	28	2.6 MB	\$0.0000
external_sheets	4	975.0 KB	\$0.0000
dbt_dev6	1	795.0 KB	\$0.0000
dbt_cloud_pr_XXXXXX_120	1	243.2 KB	\$0.0000

Partitioning and clustering

Which tables are set up to let BigQuery skip data, and which are not. Partitioning is what prunes a scan; clustering only reduces the work within what is already scanned. This section is in the report only - it is layout, not activity, and it changes when models are rebuilt rather than day to day.

Layout	Value
Tables with storage	573
Partitioned	94 (16.4%)
Clustered	153 (26.7%)
Neither	420
Bytes in partitioned tables	351.8 GB
Bytes in unpartitioned tables	42.7 GB

Unpartitioned, queried, and over 10 GB

Nothing to flag: every table above the size threshold that was queried in this window is partitioned.

Full inventory, largest first

Table	Size	Partitioned by	Clustered by	Queries
dw_prod.fact_FinancialAmounts	217.4 GB	DateKey	erp_id, LegalUnitNo	356
dbt_dev1.fact_FinancialAmounts	48.4 GB	DateKey	erp_id, LegalUnitNo	85
dw_prod.fact_RevenueAmounts	28.8 GB	DateKey	erp_id, LegalUnitNo	172
dbt_dev1.fact_RevenueAmounts	28.8 GB	DateKey	erp_id, LegalUnitNo	42
dw_prod.fact_Account_Balance	4.3 GB	DateKey	erp_id, LegalUnitNo	263
dw_prod.fact_Account_Balance_bkp_20260903	4.3 GB	DateKey	erp_id, LegalUnitNo	1
dbt_dev1.fact_Account_Balance	4.2 GB	DateKey	erp_id, LegalUnitNo	85
dw_test.fact_Account_Balance	3.5 GB	DateKey	erp_id, LegalUnitNo	5
dw_prod_stg.stg_AccountBalance	3.4 GB	-	-	149
dw_test_stg.stg_AccountBalance	3.4 GB	-	-	4
dbt_dev1_stg.stg_AccountBalance	3.4 GB	-	-	85
dw_prod_stg.stg_AccountBalance_8	3.3 GB	-	-	10
dbt_cloud_pr_XXXXXX_189.fact_Account_Balance	3.3 GB	DateKey	LegalUnitNo	0
dbt_dev3.fact_Account_Balance	3.2 GB	DateKey	LegalUnitNo	3
dbt_cloud_pr_XXXXXX_189_stg.stg_AccountBalance	3.1 GB	-	-	0
dbt_dev3_stg.stg_AccountBalance	3.0 GB	-	-	0
dw_test.fact_FinancialAmounts	1.8 GB	DateKey	erp_id, LegalUnitNo	0
dw_prod.dim_Customer	1.6 GB	-	LegalUnitNo	625
dbt_dev3_stg.stg_FinancialAmounts	1.4 GB	-	-	4
dbt_dev3.fact_FinancialAmounts	1.4 GB	DateKey	erp_id, LegalUnitNo	4
dbt_dev1_stg.stg_FinancialAmounts	1.2 GB	-	-	119
dbt_dev2_stg.stg_RevenueAmounts	1.2 GB	-	-	0
dw_prod.dim_Employees_src	1.1 GB	-	-	151
dw_prod_stg.stg_FinancialAmounts	1.1 GB	-	-	485
dbt_dev1_stg.stg_FinancialAmounts_6	1.1 GB	-	-	0

Table	Size	Partitioned by	Clustered by	Queries
dw_prod_stg.stg_FinancialAmounts_6	1.0 GB	-	-	1
dw_test_stg.stg_FinancialAmounts	1.0 GB	-	-	0
dbt_dev3.dim_Customer	953.7 MB	-	LegalUnitNo	3
dw_prod.fact_Forecast_freezed	872.8 MB	-	-	9
dbt_dev2.fact_RevenueAmounts	853.2 MB	DateKey	LegalUnitNo	0
dbt_cloud_pr_XXXXXX_189_stg.stg_FinancialAmounts	810.2 MB	-	-	0
dbt_cloud_pr_XXXXXX_189.fact_FinancialAmounts	810.2 MB	DateKey	LegalUnitNo	0
dw_prod_stg.stg_FinancialAmounts_bk	749.3 MB	-	-	1
dw_prod_stg.stg_FinancialAmounts_bk_06_07_2026	722.1 MB	-	-	1
dw_test.dim_Customer	646.3 MB	-	LegalUnitNo	13
dbt_dev1.dim_Customer	646.3 MB	-	LegalUnitNo	83
dbt_dev2_stg.stg_FinancialAmounts	640.6 MB	-	-	0
dbt_dev2.fact_FinancialAmounts	640.6 MB	DateKey	LegalUnitNo	0
dbt_dev1.fact_CustomerOpenBalanceDueDateLocked	487.2 MB	-	LegalUnitNo	53
dw_prod.fact_CustomerOpenBalanceDueDateLocked	485.0 MB	-	LegalUnitNo	53

Access review

Who holds standing access to this project, and who can elevate. Read from the live IAM policy and Privileged Access Manager at the moment this report was generated.

Identities	Count
Users	0
Service accounts	0
Groups	0
Domains	0

Privileged Access Manager entitlements

Entitlement	Grants	Max duration	State
bq-viewer-team	BigQuery Data Viewer, Custom Role, Logs Viewer	10 hours	AVAILABLE
test-to-be-removed	BigQuery Data Viewer	3 hours	AVAILABLE
bq-editor-team	BigQuery Data Editor, Custom Role, Logs Viewer	2 hours	AVAILABLE
bq-admin-team	BigQuery Admin, Custom Role, Logs Viewer	2 hours	AVAILABLE

Method and limits

Every number here comes from BigQuery metadata - INFORMATION_SCHEMA job history, table storage and column layout - plus the project's IAM policy. No table contents were read, and nothing was written.

Query cost is bytes **billed** multiplied by the per-TiB rate shown on page 1. Bytes billed is what BigQuery charges for: it includes the per-table minimum and is zero for cache hits, which is why it differs from bytes processed.

Storage cost is table bytes multiplied by the configured monthly rates. It is a monthly figure, not a figure for the reporting window.

Per-table spend counts a job's entire scan against each table the job referenced, because BigQuery does not break bytes down per table. Ranking is meaningful; summing the column is not.

Costs exclude the 1 TiB/month free query tier, streaming inserts, and any committed-use or capacity discounts. Under Editions pricing the per-TiB figures do not apply at all - slot hours are the meter, and they are on page 1.

Findings are pattern matches over truncated query text, so they can miss cases and can occasionally flag a query whose filter is expressed in a form the check does not recognise. Each one lists its examples so it can be verified.

Generating this report ran 3 metadata queries. What that itself billed is visible in the dashboard's Setup panel, under what Crucix costs.